

Document number: P2957R2
Date: 2024-10-14
Audience: SG21
Reply-to: *Andrzej Krzemieński <akrzemi1 at gmail dot com>*
Iain Sandoe <iain at sandoe dot co dot uk>
Joshua Berne <jberne4 at bloomberg dot net>
Timur Doumler <papers at timur dot audio>

Contracts and coroutines

This paper proposes how preconditions and postconditions should interact with coroutines.

Coroutines generalise regular functions in a specific manner: control can return to the caller even though the function body has not finished executing. This gives rise to some doubts as to whether the semantics of postconditions, as defined for regular functions, still apply. This paper solves this problem, and therewith addresses one of the concerns of a major tool-chain vendor expressed in [\[P3173R0\]](#).

0. Revision History

0.1. R0 → R1

- Now proposing that it is unspecified whether preconditions see the function parameters before the copying of parameters or the copies of the parameters.
- No longer proposing postconditions on coroutines.

0.2. R1 → R2

- Now requiring that preconditions see the function parameters before they are copied to the coroutine frame.
- Now proposing that postconditions on coroutines be allowed, but make it ill-formed when they use non-reference function parameters.
- We now have a reference implementation in GCC.
- Added the proposed wording.
- Reformulated the entire paper in order to highlight the definition of a *ramp function* and its implications.
- Demonstrated how *parameter captures* can enable referring to coroutine parameters in postconditions in the future.

1. Executive summary

The key observation is that, in current C++, one cannot tell from a function declaration, or call, whether the callee is a coroutine or a 'regular' function.

We reformulate the definition of a postcondition so that it does not refer to the function body.

We derive the properties for pre- and postconditions on coroutines:

- they should be allowed on declarations of functions that are later defined as coroutines,
- the program state they describe and the point of their evaluation relate to the call site,

- postconditions should not relate to the state of the coroutine or the time when it finishes.

2. Motivation

Because the start of a coroutine is conceptually no different than the start of a normal function, also the motivation for expressing a precondition is the same as for normal functions:

```
generator<int> sequence(int from, int to)
    pre (from <= to);
```

But when we return to the caller, the situation is different. The caller doesn't get the ultimate result of the operation. Instead, it gets a *tool* (a generator, a callback, an awaitable) for *advancing* the state of the of the suspended operation at a later time.

Thus, there are fewer things to express in the postcondition, less need to access function parameters (as they are often used to match against the returned value), no option to inspect the ultimate result. The only practical thing is to inspect the state of the said tool:

```
awaitable<int> cancelable_session(int id)
    post (r: is_cancelable(r));

void caller()
{
    awaitable<int> s = cancelable_session(1);
    contract_assert (is_cancelable(s));
    global_cancelable_sessions.push(std::move(s));
}
```

3. Coroutine properties

3.1. The ramp function

The C++ Standard does not clearly define the *ramp function*; however, this notion is crucial for implementing C++ coroutines in a compiler, for understanding coroutines as a user, and for us to design the semantics of postconditions.

When the compiler encounters a coroutine definition — that is, a function body with one of the coroutine keywords — it generates two things:

1. The *coroutine state (frame)*, possibly heap-allocated, storing the copies of function parameters, and tracking the the progress of the function body.
2. The *ramp function*.

The ramp function has the following properties:

- It is a normal (non-coroutine) function.
- It prepares the coroutine state.
- It executes the part of the coroutine body, until the first suspension (or till the end, if no suspension is requested).
- It returns the *return object* obtained from the coroutine promise object.

- You cannot see its body: it is compiler-generated, based on coroutine customization points.
- This is the function that the caller actually calls.

The crucial property here is that it is a normal function. Hence, whatever [\[P2900R9\]](#) has to say about pre- and postconditions for normal functions, can also be applied to the ramp function.

3.2. The normal function interface

There is no hint in a function declaration, whether it is a coroutine or not. When you invoke a coroutine, you are invoking a (normal) ramp function. Hence, the caller has no way of telling if there is a coroutine involved in the call:

```
awaitable<int> session(int id);    // may be a coroutine, may be a function

awaitable<int> default_session()  // definitely a normal function
{
    awaitable<int> s = session(0); // maybe invoking a coroutine, maybe a function
    return s;
}
```

Even though a function is returning a type that may be indicative of a coroutine implementation — an *Awaitable* or a *generator* — these are just types, and they can be as well returned by normal functions, such as factories, or forwarding functions.

3.3. Copies of function parameters

A notable thing, performed by the ramp function, is to *copy* the function parameters into the coroutine frame [\[dcl.fct.def.coroutine\]/13](#):

For a parameter of type *cv T*, the copy is a variable of type *cv T* with automatic storage duration that is direct-initialized from an *xvalue* of type *T* referring to the parameter.

Thus defined initialization can end up in performing a *move*, even if a function parameter is declared *const* by the user. This means that effectively the ramp function behaves as if its function parameters had their *const* qualification removed from the defining declaration.

4. Redefining postconditions

Before the addition of coroutines (and fibers), code in a single thread is non-interleaved: a started operation has to finish before the next one can start. Therefore, usually the definitions of pre- and postconditions make a silent assumption that we are describing a non-interleaved flow. In this view, the evaluation of the postcondition happens:

1. After the function body has finished (local automatic objects have been destroyed).
2. After the control is returned to the caller (the result object has been initialized).

In fact, for normal functions these two are the same thing.

In case of coroutines, the above two are potentially different points in time, and the "non-interleaving" definition of a postcondition cannot be maintained.

We observe that the first property above bears no relation to the caller, and a postcondition, as part of a function contract, is about describing the relation between the caller and the callee. Therefore we argue that the first property can be dropped without compromising the goal of a postcondition.

We propose the following conceptual model. Any remaining part of the coroutine body, after the ramp function returns to the caller, can be treated as a callback C returned to the caller. The caller may call C later or never, but is not directly affected by the results of calling C .

An equivalent formulation of our model is that pre- and postconditions should behave as if we wrapped the call to a coroutine function into a factory function F , declared the same contract in F , and called it instead.

4.1. Postconditions and function bodies

One effect of the proposed model is that the function body bares no connection to the predicate expressed in the postcondition:

```
awaitable<int> cancelable_session(int id)
    post (r: is_cancelable(r))
{
    int ans = co_await communicate(id);
    co_return ans;
}
```

In the above example there is nothing in the function body that would indicate that the returned r is made "cancelable".

Occasionally, the function body might give a false impression that it relates to the postcondition assertion:

```
task<int> fun(int& obj)
    // postcondition: obj >= 0
{
    // ...
    obj = 1; // evaluated long after the postcondition
    co_return;
}
```

This behavior is *not* a consequence of this proposal, but of the coroutine mechanics. Note that the notion of a "postcondition", even though not expressible as a language feature in the present C++, does exist. Authors of coroutines must already take this effect into account.

5. Inspecting coroutine parameters

The only technical challenge is about inspecting the function parameters in pre- and postcondition assertions, given that these parameters are potentially moved to the coroutine frame. This problem applies only to non-reference parameters.

Preconditions must be evaluated before this move. This is because function parameters may be used in the call to the allocation function that provides the storage for the coroutine frame. This may cause a situation surprising to the programmers: a precondition assertion sees different objects than the coroutine body. As a result, the same predicate can give different results in the two places, when the address of the object is inspected, or when the copy/move constructor of the class creates objects in a state that is different than the state of the source object.

However, we note that this surprising behavior predates contracts and is inherent to the coroutine design in C++. Even in C++23 programmers may be surprised that the parameters the coroutine body sees are not the ones that they passed as arguments.

The challenge for the postconditions is that at the point they are evaluated, the copying of the parameters had already occurred and the parameters are potentially in a moved-from state: definitely not the state that the caller intends to inspect.

We propose to make it ill-formed when a coroutine function has a postcondition that odr-uses a non-reference parameter. This decision is a direct consequence of two other decisions already made:

1. [\[P2900R9\]](#) makes the program ill-formed when a postcondition (in a normal function) references a non-`const`, non-reference function parameter.
2. The present specification of coroutines allows the implementations to move from `const` function parameters, effectively behaving as if the top-level cv-qualifications were removed from the parameters.

The rationale for requiring `const` for non-reference function parameters (for normal functions) has been provided in [\[P2466R0\]](#). Alternative approaches were considered and rejected as worse or unimplementable. They are discussed in [\[P2466R0\]](#) and, in the context of coroutines, in [\[P3387R0\]](#). We only list them here:

1. Allow the mutation of parameters, resulting in postconditions observing unintended object state.
 - That would prefer run-time surprises to compile-time surprises.
2. Making silent additional copies of referenced parameters.
 - That would only partially solve the problems for some types, while still causing run-time surprises.
3. Simply disallowing any postcondition whatsoever.
 - This seems unnecessarily harsh, given that often we only want to inspect the returned object.
4. Allow the postconditions to refer to objects, as long as their move constructor is trivial or not defined.
 - This would introduce the "spooky action at a distance" effect, where one person changes their class X and unknowingly makes somebody else's code fail to compile.
5. Have the postconditions refer to the copies of function parameters residing in the coroutine frame.
 - This is the only one specific to coroutines.
 - It is unimplementable in the current coroutine model, where user-defined promise types can choose that the coroutine is scheduled to be finished asynchronously (including the destruction of the coroutine frame) from another thread, before it initially suspends, without any sequencing related to returning from the ramp function.

[\[P3387R0\]](#) compares trade-offs for all these options, and ultimately reaches the same conclusion.

As a consequence, while for normal functions the programmer can enable a postcondition to odr-use a non-reference parameter via adding the `const` qualification to that parameter in all declarations, this will be impossible for coroutines. We expect that the compiler error that the user will see will appear only when the function body is encountered. Parsing the non-defining declarations should compile, as at this point the compiler doesn't yet know if it deals with the coroutine or not.

```
generator<int> sequence(const int limit)
    post (g : g.size() == limit);           // OK

generator<int> sequence(const int limit)    // ok to skip post-
{
    for (int i = limit; i-- != 0;)
        co_yield i;                       // Error: a coroutine
}
```

Note that this is similar to the case where the definition of a normal function does not specify the parameter odr-used in the postcondition as const.

```
generator<int> sequence(const int limit)
    post (g : g.size() == limit);           // OK

generator<int> sequence(int limit)         // Error: non-const param
{
    return sequence_impl(limit);
}
```

5.1 Future directions

[\[P2461R1\]](#) proposes a future addition to postcondition assertions that would allow the user to capture (copy) function parameters upon the function entry, so that these copies are available when evaluating the postcondition expression upon function return. Referring to such captures is safe, and the copying would be explicit. This would enable the postconditions to refer to coroutine parameters:

```
generator<int> sequence(int from, int to)
    pre (from <= to)
    post[from, to] (g : g.size() == to - from + 1);
```

6. Coroutine-specific guarantees

[\[P2932R3\]](#) argues against integrating pre- and postconditions with coroutines too early:

Currently, no concrete proposal covers the full breadth of the interface a coroutine has with its callers. Without this complete picture, we cannot yet know if pre and post will have a meaning that is correct and useful to those calling into or implementing coroutines.

In the light of a clearer description of a coroutine call as a call to the ramp function, we are confident that the model for pre- and postconditions we described is the only one possible.

While it is true that one could invent different kinds of correctness checks — like constraints on what the returned awaitable yields upon `co_await`, or testing the coroutine state at different time points — this problem is no different from checking the state of regular functions at different points of execution or checking the behavior of callbacks returned from regular functions. Yet, SG21 did not decide to hold off contracts on regular functions only because there is not a complete picture for regular functions. Any additional correctness checks, if needed, can be added on top of what we propose.

7. Proposal

We propose that preconditions on coroutines be allowed. Their evaluation is sequenced after the function parameters are initialized, and before

- the potential allocation of a coroutine frame [\[dcl.fct.def.coroutine\]/9](#),
- the initialization of the coroutine promise object [\[dcl.fct.def.coroutine\]/5.7](#).

Parameter names referenced in the precondition predicate refer to the original parameters before the move to the coroutine frame.

We propose that postconditions on coroutines be allowed, and have the following properties.

- The program is ill-formed when a coroutine postcondition odr-uses a non-reference function parameter.
- Postcondition evaluation is sequenced after the value, if any, has been returned from the ramp function to the caller, and before the function parameters (which are moved-from at that time) have been destroyed.
- The postcondition evaluation is unsequenced with respect to the termination of the evaluation of the coroutine body and the destruction of its automatic variables.

8. Wording

The proposed wording is relative to [\[P2900R9\]](#). It is equivalent to the wording provided in [\[P3387R0\]](#) and [\[P2900R10\]](#).

Modify [\[dcl.contract.func\]](#) (a section from [\[P2900R9\]](#), paragraph 6 as follows.

A ~~coroutine~~ (~~[\[dcl.fct.def.coroutine\]](#)~~), a deleted function ([\[dcl.fct.def.delete\]](#)); or a function defaulted on its first declaration ([\[dcl.fct.def.default\]](#)) may not have a *function-contract-specifier-seq*.

Modify [\[dcl.fct.def.coroutine\]](#), paragraph 5 as follows.

A coroutine behaves as if the top-level cv-qualifiers in all *parameter-declarations* in the declarator of its *function-definition* were removed, and its *function-body* were replaced by the following replacement body:

```
{
    promise-type promise promise-constructor-arguments;
    try {
        co_await promise.initial_suspend();
        function-body
    } catch ( ... ) {
        if (!initial-await-resume-called)
            throw ;
        promise.unhandled_exception();
    }
    final-suspend:
    co_await promise.final_suspend();
}
```

Drafting note: the goal is to highlight that the function non-reference parameters are modified (moved from) even if they are declared const. This, however, requires to explicitly say later that cv-qualifiers are nonetheless respected in some places.

Modify Modify [\[dcl.fct.def.coroutine\]](#), paragraph 9 as follows.

An implementation may need to allocate additional storage for a coroutine. This storage is known as the *coroutine state* and is obtained by calling a non-array allocation function

([\[basic.stc.dynamic.allocation\]](#)) as part of the replacement body. The allocation function's name is looked up by searching for it in the scope of the promise type.

- If the search finds any declarations, overload resolution is performed on a function call created by assembling an argument list. The first argument is the amount of space requested, and is a prvalue of type `std::size_t`. The lvalues $p_1 \dots p_n$ with their original cv-qualifiers are the successive arguments. If no viable function is found ([\[over.match.viable\]](#)), overload resolution is performed again on a function call created by passing just the amount of space required as a prvalue of type `std::size_t`.

Modify [\[dcl.fct.def.coroutine\]](#), paragraph 13 as follows.

When a coroutine is invoked, ~~after initializing its parameters ([\[expr.call\]](#))~~ at the beginning of the replacement body, a copy is created for each coroutine parameter. For a parameter whose original declaration was of type cv T:

- If T is a reference type, the copy is a reference of type cv T bound to the same object as the parameter.
- Otherwise, the copy is a variable of type cv T with automatic storage duration that is direct-initialized from an xvalue of type T referring to the parameter. [Note: An identifier in the function-body that names one of these parameters refers to the created copy and not the original parameter ([\[expr.prim.id.unqual\]](#)) — end note]

~~[Note: An original parameter object is never a const or volatile object ([\[basic.type.qualifier\]](#)). — end note]~~

Modify [\[intro.execution\]](#), paragraph 11 as follows.

When invoking a function *f* (whether or not the function is inline), every argument expression and the postfix expression designating *f* are sequenced before every precondition assertion of ~~the function call ([\[expr.call\]](#))~~, which in turn are sequenced before every expression or statement in the body of *f*, which in turn are sequenced before every postcondition assertion of the function call. Several contexts in C++ cause evaluation of a function call, even though no corresponding function call syntax appears in the translation unit.

Insert the following new paragraph after [\[expr.call\]](#), paragraph 8.

When control is transferred back to this function call ([\[stmt.return\]](#), [\[expr.await\]](#)), all postcondition assertions of the function call are evaluated in sequence ([\[dcl.contract.func\]](#)). [Note: This in turn is sequenced before the destruction of any function parameters. — end note]

Drafting note: this captures the return from a normal function call, as well as the "return from the ramp-function", and the end of the evaluation of the *await-expression*. [\[N4988\]](#) has the return from a coroutine call underspecified; this wording doesn't try to fix this.

Modify the paragraph inserted by [P2900R9] after [stmt.return], paragraph 3, as follows.

~~All postcondition assertions ({dcl.contract.func}) of the function call ({expr.call}) are evaluated in sequence. The destruction of all local variables within the function body is sequenced before the evaluation of any postcondition assertions.~~ [Note: Postcondition assertions of the function call ({expr.call}) are evaluated in sequence after the destruction of any local variables in scopes exited by the return statement, and are, in turn, sequenced before the destruction of function parameters. — end note]

Drafting note: this note excludes the case when the control is returned via suspending a coroutine.

9. Implementability

Currently the only compiler to implement contracts and coroutines is GCC. It implements the [P0542R5] version of contracts.

The implementation in trunk implements [[assert: _]] and [[pre: _]] as proposed in this paper. Here is a working example in Compiler Explorer: <https://godbolt.org/z/x5bTW5W6o>.

The implementation of [[post: _]] differs from the proposed semantics in one aspect: postconditions are allowed to odr-use non-reference function parameters, which results in the assertion expressions observing the moved-from state of the parameters. There are no difficulties foreseen with adding the additional required check.

The GCC implementation uses the ramp function implementation. Adding contracts on a coroutine is implemented via adding contracts to the ramp, which is no different than on any other normal function. Currently, the GCC implementation puts the runtime checks for pre- and post-conditions inside the function. This reflects that the current GCC implementation for contracts is callee-only. On-going work to prototype virtual function handling is thought to be adaptable to general caller-side contracts and to be compatible with coroutines without any special action.

10. Acknowledgments

Lewis Baker, Tom Honermann and Lisa Lippincott offered useful feedback and improved the quality of the paper.

11. References

- [P0542R5] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++", (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0542r5.html>).
- [P2461R1] — Gašper Ažman, Caleb Sunstrum, Bronek Kozicki, "Closure-Based Syntax for Contracts" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2461r1.pdf>).
- [P2466R0] — Andrzej Krzemiński, "The notes on contract annotations" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2466r0.html>).
- [P2900R9] — Joshua Berne, Timur Doumler, Andrzej Krzemiński et al., "Contracts for C++", (<https://isocpp.org/files/papers/P2900R9.pdf>).
- [P2900R10] — Joshua Berne, Timur Doumler, Andrzej Krzemiński et al., "Contracts for C++", (<https://isocpp.org/files/papers/P2900R10.pdf>).

- [P2932R3] — Joshua Berne, "A Principled Approach to Open Design Questions for Contracts" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2932r3.pdf>).
- [P3173R0] — Gabriel Dos Reis, "P2900R6 May Be Minimal, but It Is Not Viable" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3173r0.pdf>).
- [N4988] — Thomas Köppe, "Working Draft Programming Languages — C++" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/n4988.pdf>).
- [P3387R0] — Timur Doumler, Peter Bindels, Joshua Berne, "Contract assertions on coroutines" (<https://isocpp.org/files/papers/P3387R0.pdf>).